

Let's implement **all five optional extras** to let the database handle the heavy lifting. We will update the schema to include automatic timestamps, utilize SQL's `LIKE` operator, sort alphabetically, and compute real statistical summaries natively.

Since changing an existing database schema by hand can be messy, the code below includes structural updates that automatically handle adding the new timestamp columns if your database file already exists.

Choose the code block below that matches your project lane.

● Node.js Lane (Express + better-sqlite3)

Update your `server.js` file with this optimized implementation:

javascript

```
const express = require('express');
const Database = require('better-sqlite3');
const app = express();
const PORT = 3000;

app.use(express.json());

const db = new Database('tasks.db');

// Create the updated table schema including timestamp fields
db.prepare(`
  CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    done INTEGER DEFAULT 0,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
  )
`).run();

// Migration Guard: Add columns if an old database file exists without them
try {
  db.prepare("ALTER TABLE tasks ADD COLUMN created_at DATETIME DEFAULT CURRENT_TIMESTAMP")
  db.prepare("ALTER TABLE tasks ADD COLUMN updated_at DATETIME DEFAULT CURRENT_TIMESTAMP")
} catch (e) {
```

```

    // Columns already exist, skip safe migration check
  }

  // Initial seed tracking check
  const rowCount = db.prepare('SELECT COUNT(*) as count FROM tasks').get();
  if (rowCount.count === 0) {
    const insertTask = db.prepare('INSERT INTO tasks (title, done) VALUES (?, ?)');
    insertTask.run("Buy groceries", 0);
    insertTask.run("Clean the room", 1);
    insertTask.run("Study API design", 0);
  }

  // =====
  // 🔍 Extras: Search, Filter, & Alphabetical Sort
  // =====
  app.get('/tasks', (req, res) => {
    const { done, search } = req.query;

    // Use 'ORDER BY title ASC' to enforce alphabetical ordering
    let query = 'SELECT * FROM tasks WHERE 1=1';
    const params = [];

    // Filter using WHERE clause directly in the database engine
    if (done !== undefined) {
      query += ' AND done = ?';
      params.push(done === 'true' ? 1 : 0);
    }

    // Search using the SQL LIKE operator instead of a code loop
    if (search !== undefined && search.trim() !== "") {
      query += ' AND LOWER(title) LIKE ?';
      params.push(`%${search.toLowerCase()}%`);
    }

    // Apply sorting constraint
    query += ' ORDER BY title ASC';

    const rows = db.prepare(query).all(...params);

    res.json(rows.map(row => ({
      id: row.id,
      title: row.title,
      done: Boolean(row.done),
      created_at: row.created_at,

```

```

    updated_at: row.updated_at
  }));
});

// =====
// 📊 Extras: Natively Computed Statistics
// =====
app.get('/stats', (req, res) => {
  // Use COUNT functions to let the database handle math aggregates
  const total = db.prepare('SELECT COUNT(*) as count FROM tasks').get().count;
  const done = db.prepare('SELECT COUNT(*) as count FROM tasks WHERE done = 1').get().count;
  const open = db.prepare('SELECT COUNT(*) as count FROM tasks WHERE done = 0').get().count;

  res.json({ total, done, open });
});

// =====
// ⌚ Extras: Timestamp Mutations (Insert / Update)
// =====
app.post('/tasks', (req, res) => {
  const { title } = req.body;
  if (!title || title.trim() === '') return res.status(400).json({ error: "Title is required" });

  const result = db.prepare('INSERT INTO tasks (title, done) VALUES (?, 0)').run(title);
  const task = db.prepare('SELECT * FROM tasks WHERE id = ?').get(result.lastInsertRowid);

  res.status(201).json({ ...task, done: Boolean(task.done) });
});

app.put('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
  const { title, done } = req.body;

  const currentTask = db.prepare('SELECT * FROM tasks WHERE id = ?').get(taskId);
  if (!currentTask) return res.status(404).json({ error: "Task not found" });

  const finalTitle = title !== undefined ? title : currentTask.title;
  const finalDone = done !== undefined ? (done ? 1 : 0) : currentTask.done;

  // Explicitly update the updated_at timestamp flag during modification runs
  db.prepare(`
    UPDATE tasks
    SET title = ?, done = ?, updated_at = CURRENT_TIMESTAMP
    WHERE id = ?
  `).run(finalTitle, finalDone, taskId);
});

```

```

    `).run(finalTitle, finalDone, taskId);

    const updatedTask = db.prepare('SELECT * FROM tasks WHERE id = ?').get(taskId);
    res.json({ ...updatedTask, done: Boolean(updatedTask.done) });
  });

  app.listen(PORT, () => console.log(`Server on http://localhost:${PORT}`));

```

Use code with caution.

● Python Lane (FastAPI + sqlite3)

Update your `main.py` file with this optimized implementation:

python

```

import sqlite3
from fastapi import FastAPI, HTTPException, status, Query
from pydantic import BaseModel, Field
from typing import Optional

app = FastAPI()
DATABASE_FILE = "tasks.db"

class TaskCreate(BaseModel):
    title: str = Field(..., min_length=1)

class TaskUpdate(BaseModel):
    title: Optional[str] = Field(None, min_length=1)
    done: Optional[bool] = None

def init_db():
    conn = sqlite3.connect(DATABASE_FILE)
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS tasks (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            done INTEGER DEFAULT 0,

```

```

        created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
        updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
    )
    """)
conn.commit()

# Migration Guard: Add columns if an old database file exists without them
try:
    cursor.execute("ALTER TABLE tasks ADD COLUMN created_at DATETIME DEFAULT CURRENT_T
    cursor.execute("ALTER TABLE tasks ADD COLUMN updated_at DATETIME DEFAULT CURRENT_T
    conn.commit()
except sqlite3.OperationalError:
    pass # Columns already exist

cursor.execute("SELECT COUNT(*) FROM tasks")
if cursor.fetchone()[0] == 0:
    initial_tasks = [("Buy groceries", 0), ("Clean the room", 1), ("Study API design",
    cursor.executemany("INSERT INTO tasks (title, done) VALUES (?, ?)", initial_tasks)
    conn.commit()
conn.close()

init_db()

def get_db_connection():
    conn = sqlite3.connect(DATABASE_FILE)
    conn.row_factory = sqlite3.Row
    return conn

# =====
# 🔍 Extras: Search, Filter, & Alphabetical Sort
# =====
@app.get("/tasks")
def read_tasks(done: Optional[bool] = Query(None), search: Optional[str] = Query(None)):
    conn = get_db_connection()
    cursor = conn.cursor()

    query = "SELECT * FROM tasks WHERE 1=1"
    params = []

    if done is not None:
        query += " AND done = ?"
        params.append(1 if done else 0)

    if search is not None and search.strip():

```

```

        query += " AND LOWER(title) LIKE ?"
        params.append(f"%{search.lower()}%")

    # Enforce alphabetical sorting
    query += " ORDER BY title ASC"

    cursor.execute(query, params)
    rows = cursor.fetchall()
    conn.close()

    return [
        {
            "id": r["id"], "title": r["title"], "done": bool(r["done"]),
            "created_at": r["created_at"], "updated_at": r["updated_at"]
        } for r in rows
    ]

# =====
# 📊 Extras: Natively Computed Statistics
# =====
@app.get("/stats")
def get_stats():
    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT COUNT(*) FROM tasks")
    total = cursor.fetchone()[0]

    cursor.execute("SELECT COUNT(*) FROM tasks WHERE done = 1")
    done_count = cursor.fetchone()[0]

    cursor.execute("SELECT COUNT(*) FROM tasks WHERE done = 0")
    open_count = cursor.fetchone()[0]

    conn.close()
    return {"total": total, "done": done_count, "open": open_count}

# =====
# 🕒 Extras: Timestamp Mutations (Insert / Update)
# =====
@app.post("/tasks", status_code=status.HTTP_201_CREATED)
def create_task(task_input: TaskCreate):
    if not task_input.title.strip():
        raise HTTPException(status_code=400, detail={"error": "Title cannot be empty"})

```

```
conn = get_db_connection()
cursor = conn.cursor()
cursor.execute("INSERT INTO tasks (title, done) VALUES (?, 0)", (task_input.title,))
conn.commit()
new_id = cursor.lastrowid

cursor.execute("SELECT * FROM tasks WHERE id = ?", (new_id,))
row = cursor.fetchone()
conn.close()
return dict(row)

@app.put("/tasks/{id}")
def update_task(id: int, task_input: TaskUpdate):
    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM tasks WHERE id = ?", (id,))
    row = cursor.fetchone()
    if row is None:
        conn.close()
        raise HTTPException(status_code=404, detail={"error": "Task not found"})

    final_title = task_input.title if task_input.title is not None else row["title"]
    final_done = 1 if (task_input.done if task_input.done is not None else bool(row["done"]

    # Update title, done flag, and force update timestamp calculation
    cursor.execute("""
        UPDATE tasks
        SET title = ?, done = ?, updated_at = CURRENT_TIMESTAMP
        WHERE id = ?
    """, (final_title, final_done, id))
    conn.commit()

    cursor.execute("SELECT * FROM tasks WHERE id = ?", (id,))
    updated_row = cursor.fetchone()
    conn.close()
    return dict(updated_row)
```

Use code with caution.



Step 2: Append Observations to `README.md`

Open your project `README.md` file and document your experience updating the active table shape to finalize the experiment section:

markdown

```
## 💎 Optional Experimentation Additions
```

```
I implemented native database search queries (`LIKE`), query state filters, alphabetical r
```

```
### Reflection on Schema Shapes (Timestamps Extra)
```

```
Changing the database table structure by adding new timestamp fields felt highly disruptiv
```

Use code with caution.



Step 3: Git Commit Progress Track

Save your final production updates to your version log layout history using your shell:

bash

```
git add server.js main.py README.md
```

```
git commit -m "Extras: SQL sorting, text pattern matching, real aggregates, and database s
```

```
git push origin main
```